

LAPORAN PRAKTIKUM 3

IOT 2 Modul 3



Disusun Oleh:

Nama : Reza Aurelia Saputri

NPM : 25782080

Prodi : TRI C

Dosen:

Agus Ambarwari, S.Pd., M.Kom.

PROGRAM STUDI TEKNOLOGI REKAYASA INTERNET

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI LAMPUNG

2025/2026

PRATIKUM 1

Modul 3.5.1: NodeMCU sebagai *Web Server* (Menyajikan HTML & Sensor)

1. Tujuan Praktikum

- Memahami cara menghubungkan NodeMCU ESP8266 ke jaringan Wi-Fi lokal dalam mode *Station* (STA).
- Mampu memprogram NodeMCU ESP8266 sebagai *Web Server* yang menyajikan antarmuka halaman HTML dinamis.
- Mampu membaca data sensor suhu (DHT11) dan mengendalikan aktuator (LED/Relay) melalui halaman *web*.

2. Alat dan Bahan

- *Board* NodeMCU ESP8266 (1 buah)
- Kabel Micro USB (1 buah)
- *Breadboard* / Papan Rangkaian (1 buah)
- Sensor Suhu & Kelembapan DHT11 (1 buah)
- LED 5mm (1 buah)
- Resistor 220 Ohm (1 buah)
- Kabel *Jumper* Secukupnya

3. Skema Rangkaian (Wiring Pin)

- **Sensor DHT11:**
 - Pin VCC → Pin 3V3 NodeMCU
 - Pin GND → Pin GND NodeMCU
 - Pin DATA/OUT → Pin D4 (GPIO 2) NodeMCU
- **LED:**
 - Kaki Anoda (+) → Pin D6 (GPIO 12) NodeMCU
 - Kaki Katoda (-) → Resistor 220 Ohm → Pin GND NodeMCU

```

1  #include <ESP8266WiFi.h>
2  #include <ESP8266WebServer.h>
3  #include <DHT.h>
4
5  const char* ssid = "Galaxy S24 FE C2D6";
6  const char* password = "sherlyza";
7
8  ESP8266WebServer server(80);
9
10 const byte dhtPin = 2;
11 const byte relayPin = 12;
12 DHT dht(dhtPin, DHT22);
13
14 const char indexHtml[] PROGMEM = R"(...)";

```

Output Serial Monitor X

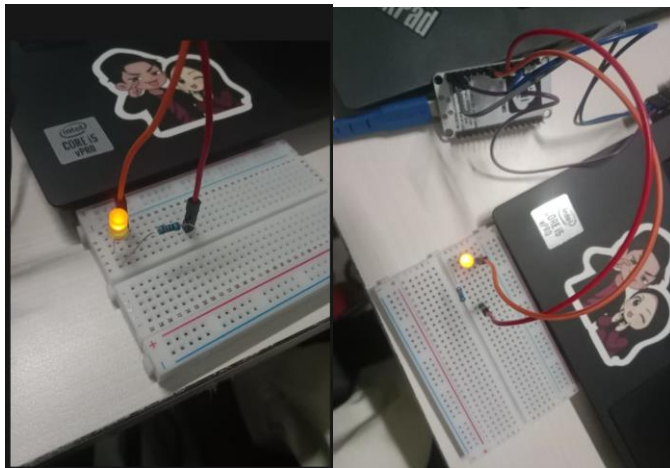
Message (Enter to send message to "NodeMCU 1.0 (ESP-12E Module)" on "COM4")

New Line 115200 baud

```

10.175.146.200
.....
IP Address Server Anda:
10.175.146.200

```



5. Langkah Percobaan & Analisis

- NodeMCU berhasil diset sebagai *Web Server* pada Port 80. Ketika diakses via *browser*, halaman web berhasil menampilkan suhu ruangan secara dinamis serta mampu mengubah status LED menjadi menyala atau padam melalui perintah *routing* HTTP (/relay/on dan /relay/off).

PRAKTIKUM 2: NodeMCU sebagai *HTTP Client* (Meminta Layanan)

1. Tujuan Praktikum

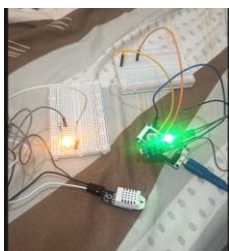
- Memahami cara memprogram NodeMCU ESP8266 sebagai **HTTP Client** yang proaktif mengirimkan permintaan ke perangkat lain di jaringan lokal.
- Mengimplementasikan pembacaan sensor analog (LDR) sebagai pemicu (*trigger*) pengiriman perintah jarak jauh menggunakan metode HTTP GET.

2. Alat dan Bahan

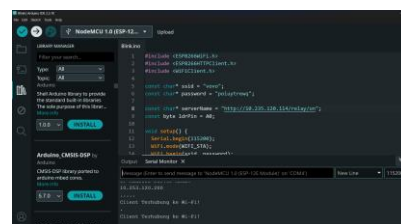
- *Board* NodeMCU ESP8266 (1 buah)
- Kabel Micro USB (1 buah)
- *Breadboard* (1 buah)
- Sensor Cahaya LDR (1 buah)
- Resistor 10k Ohm (1 buah)
- Kabel *Jumper* Secukupnya

3. Skema Rangkaian (Wiring Pin)

- **Sensor LDR & Pembagi Tegangan:**
 - Salah satu kaki LDR → Pin 3V3 NodeMCU
 - Kaki LDR lainnya tersambung dengan Resistor 10k Ohm → Pin A0 (Analog Input) NodeMCU
 - Sisa kaki Resistor 10k Ohm → Pin GND NodeMCU



ESP8266 Web Server
Suhu Saat Ini: 31.20 Celcius
Kendali Relay



5. Langkah Percobaan & Analisis

- NodeMCU bertindak sebagai *Client* yang terus memantau intensitas cahaya melalui sensor LDR. Ketika sensor ditutup (kondisi gelap dengan nilai sensor < 300), *Client* secara otomatis melakukan permintaan HTTP GET ke alamat IP *Server* (Praktikum 1) untuk menyalakan lampu LED dari jarak jauh. Respon status kode 200 pada Serial Monitor menandakan komunikasi berhasil dilakukan.

3.6 Latihan

Latihan 1 (Pemahaman Jaringan)

- **Pertanyaan:** Berdasarkan wawasan dari subbab 3.3.1, sebutkan secara logis mengapa kita menggunakan mode jaringan Station (STA) pada kedua praktikum di atas, dan bukan mode Access Point (AP)?
- **Jawaban:** Kita menggunakan mode **Station (STA)** karena NodeMCU bertindak sebagai perangkat klien yang bergabung ke dalam jaringan Wi-Fi lokal yang sudah ada (yaitu *router* atau *hotspot* vovo). Dengan berada di jaringan lokal yang sama dengan laptop atau HP pengguna, NodeMCU bisa mendapatkan alamat IP lokal yang valid agar halaman web server di dalamnya dapat diakses dengan mudah oleh perangkat lain dalam satu jaringan tersebut. Jika menggunakan mode *Access Point* (AP), NodeMCU harus memancarkan sinyal Wi-Fi sendiri, sehingga perangkat kita harus keluar dari internet utama dan kerepotan berpindah-pindah jaringan.

Latihan 2 (Modifikasi Antarmuka & Telemetry)

- **Pertanyaan:** Tuliskan potongan baris kode HTML dan fungsi `handleRoot()` yang Anda ubah untuk menampilkan kelembapan dan tombol dinamis tunggal (*toggle button*).
- **Jawaban:** Berikut adalah potongan kode modifikasi HTML dengan tambahan `%HUMIDITY%`, `%RELAY_BUTTON%`, serta fungsi `handleRoot()` yang diperbarui:

ESP8266 Web Server

Suhu Saat Ini: **31.00** Celcius

Kelembapan: **75.00** %

Kendali Relay

NYALAKAN (ON)

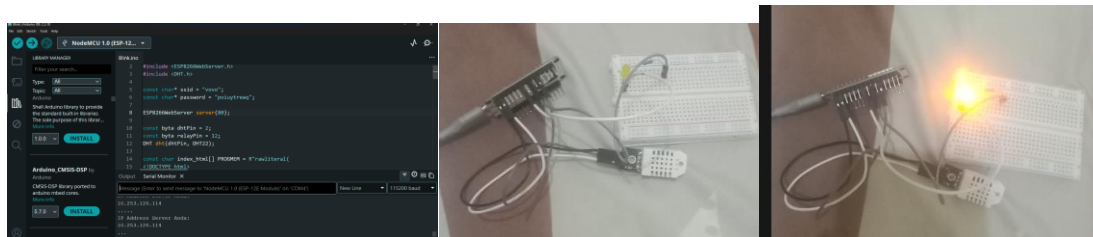
ESP8266 Web Server

Suhu Saat Ini: **31.10** Celcius

Kelembapan: **74.90** %

Kendali Relay

MATIKAN (OFF)



Latihan 3 (Analisis Kode Client)

- **Pertanyaan:** Analisislah apa yang akan dicetak oleh Serial Monitor Anda jika ternyata NodeMCU Server teman Anda mendadak kehilangan daya (*offline*)?
- **Jawaban:** Jika NodeMCU Server kehilangan daya (*offline*), Serial Monitor pada sisi Client akan mencetak kode respons HTTP gagal atau kesalahan koneksi, seperti **connection failed, error code**, atau nilai status HTTP non-200 (misalnya *connection timed out*), karena klien tidak dapat melakukan *handshake* TCP/IP maupun mengirim permintaan GET ke alamat IP server yang mati.

4. Latihan 4 (Pengembangan Algoritma)

- **Pertanyaan:** Bagaimana bentuk blok logika (*conditional block*) pada sisi Client agar pengiriman permintaan HTTP GET dilakukan HANYA JIKA sensor DHT mendeteksi suhu melebihi 35°C?
- **Jawaban:** Menggunakan blok validasi kondisi seperti berikut:

2. Jawaban Analisis Arsitektur (Evaluasi Logis)

Anda dapat menyalin bagian ini langsung ke dalam laporan praktikum Anda:

Analisis Kelemahan Metode *Meta Refresh* HTTP Konvensional pada *Real-Time Dashboard*:

1. **Terjadinya Efek Kedip Layar (*Flickering*):** Setiap interval 5 detik habis, *browser* akan memuat ulang seluruh halaman secara penuh (*full page reload*). Hal ini menyebabkan layar *browser* berkedip (biasanya kilatan putih sesaat), yang membuat pengalaman pengguna (*User Experience*) menjadi kurang nyaman dan tidak terlihat profesional untuk standar sistem *real-time*.
2. **Pemborosan *Bandwidth* dan Sumber Daya:** Karena menggunakan *full page reload*, setiap 5 detik *browser* harus mengunduh ulang seluruh komponen halaman—mulai dari struktur HTML, pengaturan CSS gaya, hingga teks statis—meskipun yang sebenarnya mengalami perubahan data hanyalah angka suhu dan kelembapan saja. Ini kurang efisien bagi mikrokontroler dengan memori terbatas seperti ESP8266.
3. **Potensi Bentrokan Waktu Eksekusi (*Race Condition* / *Timing Conflict*):** Jika pengguna mencoba mengeklik tombol **ON** / **OFF** (kendali LED) secara bersamaan atau bertepatan persis dengan detik ke-5 saat *browser* sedang melakukan *auto-refresh*, perintah klik bisa terganggu, gagal terkirim, atau terjadi pembatalan permintaan (*request timeout*), sehingga tombol harus diklik ulang.
4. **Solusi Modern (Sebagai Pembanding):** Dalam industri IoT profesional, metode *Meta Refresh* tradisional ini sudah ditinggalkan dan digantikan dengan teknologi **AJAX (Asynchronous JavaScript and XML)** atau **WebSockets**. Dengan teknologi tersebut, pembaruan data sensor dilakukan di latar belakang (*background*) secara parsial tanpa perlu memuat ulang seluruh halaman web, sehingga bebas dari kedip layar.

